



Python Programming Fundamentals

While Loops

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is a While Loop?
2. While Loop Syntax
3. Example — Countdown Timer
4. Infinite Loops
5. Break and Continue
6. The Do-While Pattern in Python
7. Accumulation with While Loops



Outline

- 1. What is a While Loop?**
2. While Loop Syntax
3. Example — Countdown Timer
4. Infinite Loops
5. Break and Continue
6. The Do-While Pattern in Python
7. Accumulation with While Loops



1. What is a While Loop?

A **while loop** repeats a block of code **as long as** a condition remains True.



1. What is a While Loop?

A **while loop** repeats a block of code **as long as** a condition remains True.

Use a while loop when:

- You don't know in advance how many times to repeat
- You want to keep asking for input until valid
- You want to retry something until it succeeds



1. What is a While Loop?

A **while loop** repeats a block of code **as long as** a condition remains True.

Use a while loop when:

- You don't know in advance how many times to repeat
- You want to keep asking for input until valid
- You want to retry something until it succeeds

while loop	for loop
Condition-driven	Collection/range-driven
Unknown iterations	Known iterations
Risk of infinite loop	Always terminates (usually)



Outline

1. What is a While Loop?
- 2. While Loop Syntax**
3. Example — Countdown Timer
4. Infinite Loops
5. Break and Continue
6. The Do-While Pattern in Python
7. Accumulation with While Loops



2. While Loop Syntax

```
while condition:  
    # body – executed while condition is True  
    # must eventually make condition False!
```




2. While Loop Syntax



2. While Loop Syntax

```
while condition:  
    # body – executed while condition is True  
    # must eventually make condition False!
```

Simple example — countdown:

```
count = 5  
  
while count > 0:  
    print(count)  
    count -= 1      # IMPORTANT: update the variable!  
  
print("Blast off!")
```

Output:



2. While Loop Syntax

```
5  
4  
3  
2  
1  
Blast off!
```



Outline

1. What is a While Loop?
2. While Loop Syntax
- 3. Example — Countdown Timer**
4. Infinite Loops
5. Break and Continue
6. The Do-While Pattern in Python
7. Accumulation with While Loops



3. Example — Countdown Timer

Let's trace through the execution step by step:

```
count = 3
```

```
while count > 0:           # check: is 3 > 0? Yes → enter loop
    print(count)           # prints 3
    count -= 1             # count is now 2

                           # check: is 2 > 0? Yes → enter loop
                           # prints 2, count becomes 1

                           # check: is 1 > 0? Yes → enter loop
                           # prints 1, count becomes 0

                           # check: is 0 > 0? No → exit loop

print("Done!")
```



3. Example — Countdown Timer



3. Example — Countdown Timer

Let's trace through the execution step by step:

```
count = 3
```

```
while count > 0:           # check: is 3 > 0? Yes → enter loop
    print(count)           # prints 3
    count -= 1             # count is now 2

                           # check: is 2 > 0? Yes → enter loop
                           # prints 2, count becomes 1

                           # check: is 1 > 0? Yes → enter loop
                           # prints 1, count becomes 0

                           # check: is 0 > 0? No → exit loop

print("Done!")
```



3. Example — Countdown Timer

Key rule: the loop body must always eventually make the condition False — otherwise you get an **infinite loop**.



Outline

1. What is a While Loop?
2. While Loop Syntax
3. Example — Countdown Timer
- 4. Infinite Loops**
5. Break and Continue
6. The Do-While Pattern in Python
7. Accumulation with While Loops



4. Infinite Loops

An **infinite loop** runs forever because the condition never becomes False.

```
# DANGER – infinite loop!
x = 1
while x > 0:
    print(x)
    x += 1      # x keeps growing, never reaches 0
```



4. Infinite Loops



4. Infinite Loops

An **infinite loop** runs forever because the condition never becomes False.

```
# DANGER – infinite loop!
x = 1
while x > 0:
    print(x)
    x += 1      # x keeps growing, never reaches 0
```

How to stop an infinite loop: press Ctrl+C in the terminal.



4. Infinite Loops



4. Infinite Loops

An **infinite loop** runs forever because the condition never becomes False.

```
# DANGER – infinite loop!
x = 1
while x > 0:
    print(x)
    x += 1      # x keeps growing, never reaches 0
```

How to stop an infinite loop: press Ctrl+C in the terminal.

Intentional infinite loops use break to exit:

```
while True:                # deliberately infinite
    user_input = input("Enter 'quit' to stop: ")
    if user_input == "quit":
        break               # exit the loop
    print(f"You said: {user_input}")
```



4. Infinite Loops

```
print("Goodbye!")
```



Outline

1. What is a While Loop?
2. While Loop Syntax
3. Example — Countdown Timer
4. Infinite Loops
- 5. Break and Continue**
6. The Do-While Pattern in Python
7. Accumulation with While Loops



5. Break and Continue

break — immediately exits the loop:

```
number = 1
while number <= 100:
    if number % 7 == 0:
        print(f"First multiple of 7: {number}")
        break          # stop as soon as we find it
    number += 1
```



5. Break and Continue

break — immediately exits the loop:

```
number = 1
while number <= 100:
    if number % 7 == 0:
        print(f"First multiple of 7: {number}")
        break          # stop as soon as we find it
    number += 1
```

continue — skips the rest of this iteration, goes back to the condition check:

```
number = 0
while number < 10:
    number += 1
    if number % 2 == 0:
        continue      # skip even numbers
    print(number)      # prints 1, 3, 5, 7, 9
```



Outline

1. What is a While Loop?
2. While Loop Syntax
3. Example — Countdown Timer
4. Infinite Loops
5. Break and Continue
- 6. The Do-While Pattern in Python**
7. Accumulation with While Loops



6. The Do-While Pattern in Python

Python has no `do...while` statement, but we can simulate it:

```
# do...while equivalent – always runs at least once
while True:
    age = int(input("Enter your age: "))
    if age >= 0:
        break
    print("Age cannot be negative. Try again.")

print(f"Your age is {age}")
```



6. The Do-While Pattern in Python



6. The Do-While Pattern in Python

Python has no `do...while` statement, but we can simulate it:

`do...while` equivalent – always runs at least once

```
while True:
```

```
    age = int(input("Enter your age: "))
```

```
    if age >= 0:
```

```
        break
```

```
    print("Age cannot be negative. Try again.")
```

```
print(f"Your age is {age}")
```

Input validation pattern — very common in real programs:

```
while True:
```

```
    choice = input("Enter Y or N: ").upper()
```

```
    if choice in ["Y", "N"]:
```

```
        break
```



6. The Do-While Pattern in Python

```
print("Invalid choice. Please enter Y or N.")

if choice == "Y":
    print("You said yes!")
else:
    print("You said no!")
```



Outline

1. What is a While Loop?
2. While Loop Syntax
3. Example — Countdown Timer
4. Infinite Loops
5. Break and Continue
6. The Do-While Pattern in Python
- 7. Accumulation with While Loops**



7. Accumulation with While Loops

```
# Sum numbers entered by user until they type 0
```

```
total = 0
```

```
count = 0
```

```
print("Enter numbers to sum (0 to finish):")
```

```
while True:
```

```
    num = float(input("Number: "))
```

```
    if num == 0:
```

```
        break
```

```
    total += num
```

```
    count += 1
```

```
if count > 0:
```

```
    average = total / count
```

```
    print(f"Sum: {total}")
```



7. Accumulation with While Loops

```
print(f"Count: {count}")
print(f"Average: {average:.2f}")
else:
    print("No numbers entered.")
```



Thanks for Watching - Any questions?

